

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS:

Total Marks: 30

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

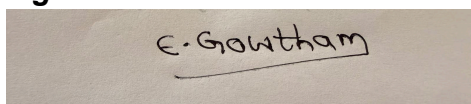
Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

I Enamala Gowtham /192311190 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:A photograph of a piece of paper with the handwritten signature "E. Gowtham" in black ink.**RUBRICS FOR CALCULATION:**

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1 — Online Student Registration System

1. Scenario

A college wants an online student registration system where students enter their details through a web form. The submitted data is processed by a **Java Servlet**. The servlet should support both **GET** and **POST** methods.

2. Servlet-Based Architecture

The architecture consists of:

1. **Client/Browser** – Student enters registration details.
2. **Web Form** – Collects student information such as name, register number, email, and department.
3. **Web Server/Application Server** – Receives the HTTP request.
4. **Servlet Container** – Runs and manages the Java Servlet.
5. **Servlet** – Processes the submitted data using `doGet()` or `doPost()`.
6. **Response** – Servlet sends the result back to the browser.

Flow:



3. HTML Registration Form

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Registration</title>
</head>
<body>

<h2>Student Registration</h2>
<form action="register" method="post">
  Name:
  <input type="text" name="name"><br><br>
  Register Number:
  <input type="text" name="regno"><br><br>
  Email:
  <input type="email" name="email"><br><br>
  Department:
```

```

        <input type="text" name="department"><br><br>
        <input type="submit" value="Register">
    </form>
</body>
</html>

```

4. Servlet Implementation

```

import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class RegistrationServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        String name = request.getParameter("name");
        String regno = request.getParameter("regno");
        String email = request.getParameter("email");
        String department = request.getParameter("department");
        out.println("<h2>Student Registration using GET</h2>");
        out.println("Name: " + name + "<br>");
        out.println("Register Number: " + regno + "<br>");
        out.println("Email: " + email + "<br>");
        out.println("Department: " + department);
    }

    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        String name = request.getParameter("name");
        String regno = request.getParameter("regno");
        String email = request.getParameter("email");
        String department = request.getParameter("department");

        out.println("<h2>Student Registration using POST</h2>");
        out.println("Registration Successful<br><br>");
        out.println("Name: " + name + "<br>");
        out.println("Register Number: " + regno + "<br>");
        out.println("Email: " + email + "<br>");

        out.println("Department: " + department);
    }
}

```

5. Servlet Lifecycle

The servlet lifecycle contains three major methods specified in the assessment: `init()`, `service()`, and `destroy()`.

`init()`

- Called only once when the servlet is created.
- Used for initialization.
- Example: establishing resources or reading configuration.

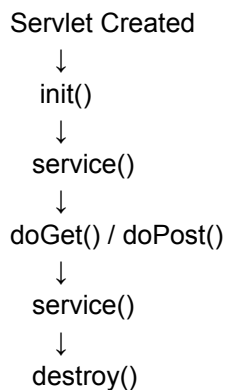
`service()`

- Called whenever a client sends a request.
- It processes the request.
- Depending on the HTTP method, it invokes `doGet()` or `doPost()`.

`destroy()`

- Called before the servlet is removed from memory.
- Used for cleanup.
- Example: closing resources.

Lifecycle:



6. GET vs POST

GET	POST
Data is sent through URL	Data is sent in request body
Data can appear in browser history	Data is not normally displayed in URL
Suitable for retrieving data	Suitable for submitting data
Less suitable for sensitive information	More appropriate for sensitive form data
Limited URL length	Can handle larger request data

7. Decision

For **student registration**, **POST is more appropriate**, especially when submitting sensitive or private information.

POST should be used because the form data is placed in the HTTP request body rather than being exposed directly in the URL. However, **POST alone does not provide encryption**; HTTPS should also be used for protecting sensitive information.

Question 2 — Login and Session Management

The scenario requires users to log in and maintain their session across multiple pages. The assessment specifically asks for **HttpSession and Cookies**, along with a comparison of session tracking and cookies.

1. Session Management

Session management allows the server to identify a user across multiple HTTP requests.

Example:

Login



Authentication



Create Session



Store User Information



Page 1



Page 2



Page 3



Logout



Session Destroyed

2. Login Servlet Using HttpSession

```
import java.io.*;
```

```
import jakarta.servlet.*;
```

```
import jakarta.servlet.http.*;
```

```
public class LoginServlet extends HttpServlet {
```

```

protected void doPost(HttpServletRequest request,
                        HttpServletResponse response)
                        throws ServletException, IOException {

    String username = request.getParameter("username");
    String password = request.getParameter("password");
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();

    if (username.equals("admin") && password.equals("1234")) {
        HttpSession session = request.getSession();
        session.setAttribute("username", username);
        out.println("<h2>Login Successful</h2>");
        out.println("<a href='home'>Go to Home</a>");
    } else {
        out.println("<h2>Invalid Username or Password</h2>");
    }
}
}

```

3. Accessing Session on Another Page

```

import java.io.*;

import jakarta.servlet.*;

import jakarta.servlet.http.*;

public class HomeServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
                        throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        HttpSession session = request.getSession(false);

```



```

if (session != null) {

    String username =

        (String) session.getAttribute("username");

    out.println("<h2>Welcome " + username + "</h2>");

    out.println("<p>You are logged in.</p>");

} else {

    out.println("<h2>Please Login First</h2>");

}

}

}

```

4. Logout

```

import java.io.*;

import jakarta.servlet.*;

import jakarta.servlet.http.*;

public class LogoutServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,

        HttpServletResponse response)

        throws ServletException, IOException {

        HttpSession session = request.getSession(false);

        if (session != null) {

            session.invalidate();

        }

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        out.println("<h2>You have been logged out.</h2>");

    }

}

```

5. Cookies

A cookie is a small piece of information stored by the browser.

Creating a Cookie

```
Cookie cookie = new Cookie("username", "admin");  
  
cookie.setMaxAge(60 * 60);  
  
response.addCookie(cookie);
```

Reading a Cookie

```
Cookie[] cookies = request.getCookies();  
  
if (cookies != null) {  
    for (Cookie cookie : cookies) {  
        if (cookie.getName().equals("username")) {  
            String username = cookie.getValue();  
            System.out.println(username);  
        }  
    }  
}
```

6. HttpSession vs Cookies

Feature	HttpSession	Cookies
Stored mainly	Server	Client/browser
Data security	Generally better	More exposed to client
Data size	More suitable for session data	Limited
Server memory	Uses server resources	Does not store session data on server

Identification	Session ID identifies session	Cookie identifies/stores information
Example	Login session	Remember username

7. How Session Tracking Works

1. User logs into the website.
2. Server authenticates the username and password.
3. Server creates an `HttpSession`.
4. A unique session ID is associated with the user.
5. The browser sends the session ID with subsequent requests.
6. Server uses the ID to find the corresponding session.
7. User can access multiple pages without logging in repeatedly.
8. When the user logs out, the session is invalidated.

8. Decision

For maintaining an authenticated login session, **`HttpSession` is the better choice** because important session information can remain on the server. Cookies can be used to help identify the session or store non-sensitive preferences.

Question 3 — E-Commerce Product Database

The scenario requires an e-commerce application to **store and retrieve product details using JDBC and SQL queries**. The assessment asks for JDBC architecture, database connectivity steps, insertion, retrieval, and exception handling.

1. JDBC Architecture

JDBC stands for **Java Database Connectivity**.

It allows Java applications to communicate with databases.

Java Application



JDBC API



JDBC Driver



Database

Main Components

1. **Java Application** – Web application or servlet.
2. **JDBC API** – Provides classes and interfaces for database operations.
3. **JDBC Driver** – Connects Java applications to a particular database.
4. **Database** – Stores product information.

2. Database Table

Example MySQL table:

```
CREATE DATABASE ecommerce;
```

```
USE ecommerce;
```

```
CREATE TABLE product (
```

```
    id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    name VARCHAR(100),
```

```
    price DOUBLE,
```

```
    quantity INT
```

```
);
```

3. JDBC Steps

The main steps are:

1. Import JDBC packages.
2. Load/register the JDBC driver.
3. Establish database connection.
4. Create a PreparedStatement.
5. Execute SQL query.
6. Process the result.
7. Close database resources.
8. Handle exceptions.

4. Insert Product Data

```
import java.sql.*;

public class InsertProduct {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/ecommerce";

        String user = "root";

        String password = "";

        String sql = "INSERT INTO product(name, price, quantity) VALUES (?, ?, ?)";

        try {

            Connection con =

                DriverManager.getConnection(url, user, password);

            PreparedStatement ps = con.prepareStatement(sql);

            ps.setString(1, "Laptop");

            ps.setDouble(2, 55000);

            ps.setInt(3, 10);

            int result = ps.executeUpdate();

            if (result > 0) {

                System.out.println("Product inserted successfully");

            }

            ps.close();

            con.close();

        }

    }

}
```

```

    } catch (SQLException e) {

        System.out.println("Database Error: " + e.getMessage());

    }

}

}

```

5. Retrieve Product Data

```

import java.sql.*;

public class DisplayProduct {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/ecommerce";

        String user = "root";

        String password = "";

        String sql = "SELECT * FROM product";

        try {

            Connection con =

                DriverManager.getConnection(url, user, password);

            Statement st = con.createStatement();

            ResultSet rs = st.executeQuery(sql);

            while (rs.next()) {

                System.out.println("ID: " +

                    rs.getInt("id"));

                System.out.println("Name: " +

                    rs.getString("name"));

                System.out.println("Price: " +

                    rs.getDouble("price"));

                System.out.println("Quantity: " +

                    rs.getInt("quantity"));
            }
        }
    }
}

```

```

        System.out.println("-----");
    }

    rs.close();

    st.close();

    con.close();

} catch (SQLException e) {

    System.out.println("Database Error: " +

        e.getMessage());

}

}

}

```

6. Exception Handling

JDBC operations can generate `SQLException`. Therefore, database code should be placed inside a try-catch block.

```

try {

    Connection con =

        DriverManager.getConnection(url, user, password);

    // Database operations

} catch (SQLException e) {

    System.out.println("Database Error: " + e.getMessage());

}

```

This prevents the application from terminating unexpectedly and provides useful information about database errors.

7. Overall E-Commerce Flow

Customer



Web Application



Servlet



JDBC API



JDBC Driver



MySQL Database



Product Data



Servlet



Web Browser

Conclusion

Java Servlets for server-side request processing, session management for maintaining authenticated users, and JDBC for database connectivity.